Introduction to the Course

Prof. Dr. Gerardo Flores

January 2026

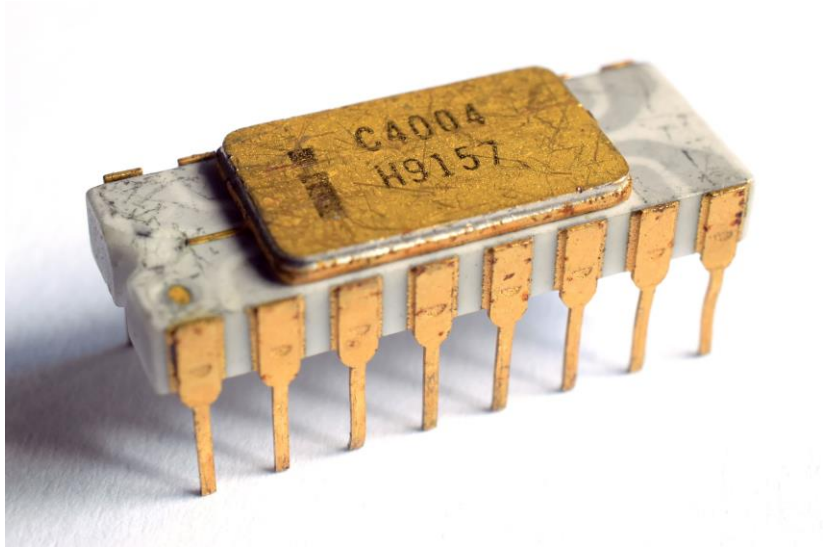
Microprocessor Systems (CSCE 3340 - SENG 3345)

TAMIU

by Federico Faggin

The Making of the First Microprocessor

The Intel 4004 CPU-on-a-chip was developed under pressure on an extremely tight schedule—and it worked.



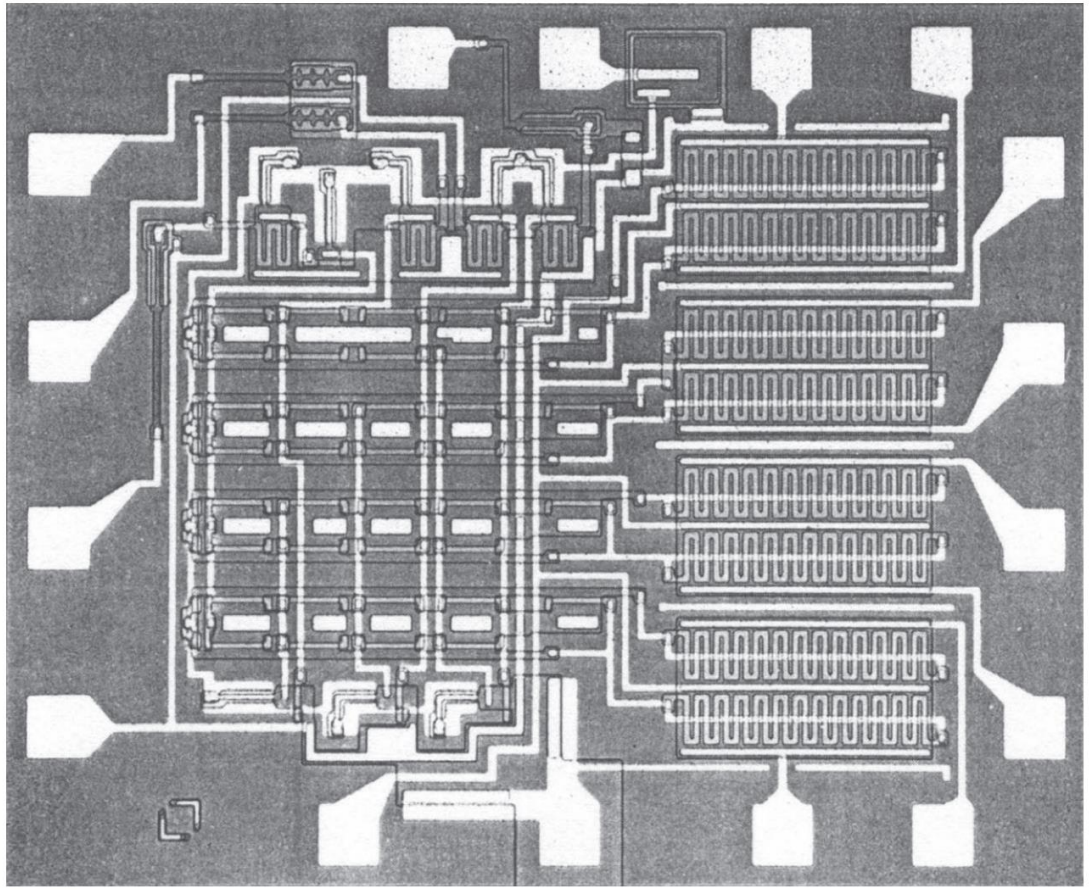
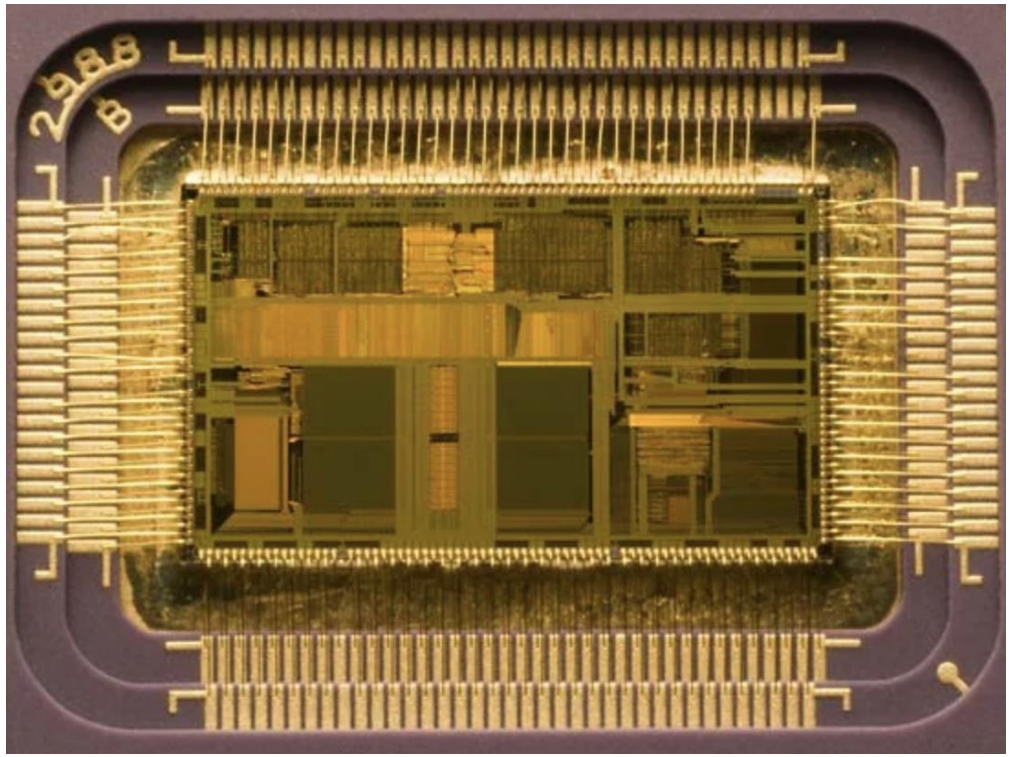


FIGURE 1: The Fairchild 3708, the world's first commercial self-aligned gate MOS IC, employing silicon gate technology and available in the market at the end of 1968.

https://ieeexplore.ieee.org/abstract/document/4776530

F. Faggin, "The Making of the First Microprocessor," in *IEEE Solid-State Circuits Magazine*, vol. 1, no. 1, pp. 8-21, Winter 2009, doi: 10.1109/MSSC.2008.930938.

Core of an Intel 80486DX2 microprocessor



32-bit microprocessor model released in 1992

https://youtu.be/Knd-U-avG0c?si=fzcHXzLdBDToMVKQ

A **CPU (Central Processing Unit)** is the functional unit of a computer responsible for executing instructions, performing calculations, and coordinating tasks, often implemented physically as part of a microprocessor.

Difference between Microprocessors and Microcontrollers

- **Microprocessor:**
  - Focused on computation and data processing.
  - Requires external memory and peripherals.
  - Used in high-performance systems like PCs and servers.
- **Microcontroller:**
  - A complete system on a chip (processor, memory, peripherals).
  - Designed for specific embedded applications.
  - Ideal for control tasks in electronics, automation, and IoT.

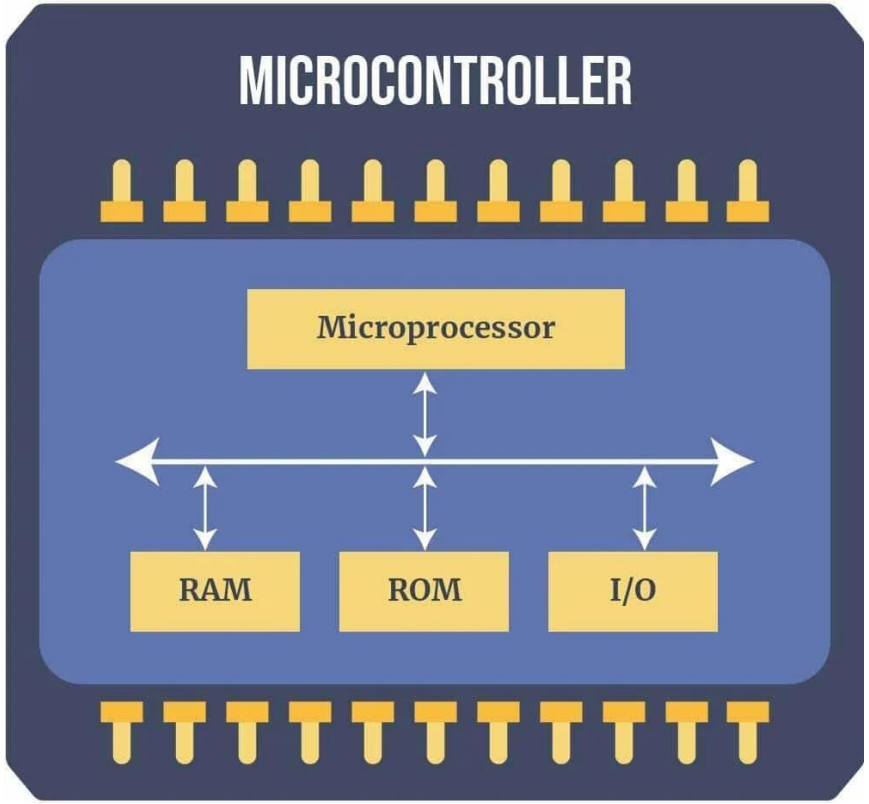
Microprocessors focus on theory and architecture, while microcontrollers integrate practical applications with peripherals.

Both circuits contain CPUs, however, microcontrollers also integrate memory, input/output (I/O) components and other varied peripherals.

A CPU is like an "engine" that drives central processing.
A microprocessor is like a "complete engine" that includes the engine (CPU) along with other complementary systems (such as the injection system, brakes, etc.) in a single package.

The key difference between these two units is that microcontrollers combine all the necessary elements of a microcomputer system onto a single piece of hardware.

Microcontrollers do not require additional peripherals or complex operating systems to function, while microprocessors do.



Key components of a microprocessor

Modern microprocessors combine millions of small transistors, resistors and diodes assembled on a semiconductor material to create the key components of a CPU.

•**Arithmetic logic unit (ALU):** The main logic unit of the CPU, this component executes logical operations including mathematical calculations and data comparisons.

•**Control unit (CU):** The CU circuit interprets instructions and initiates their execution, directing the basic operations of the processor.

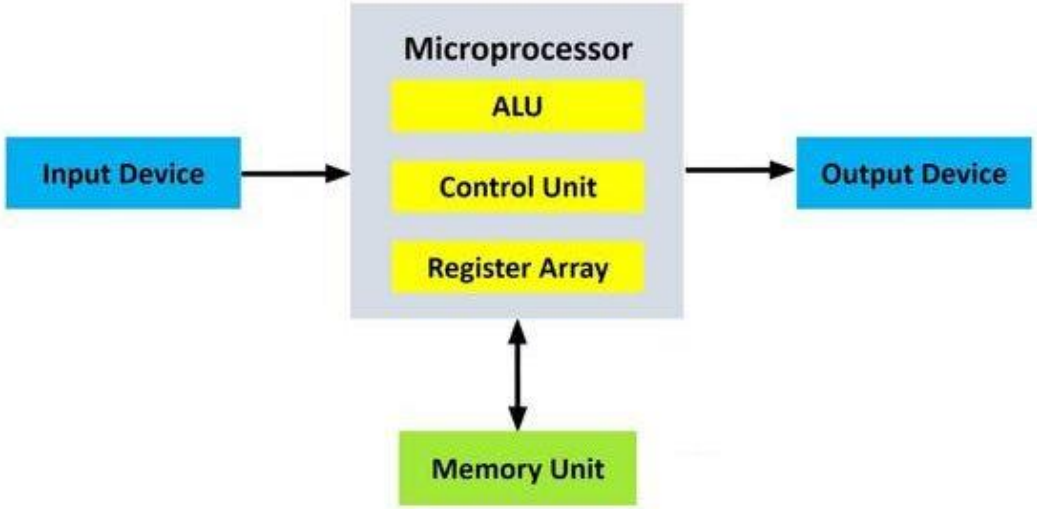
•**Registers:** Small, fast memory storage used by a CPU to temporarily hold data and instructions during computational processes.

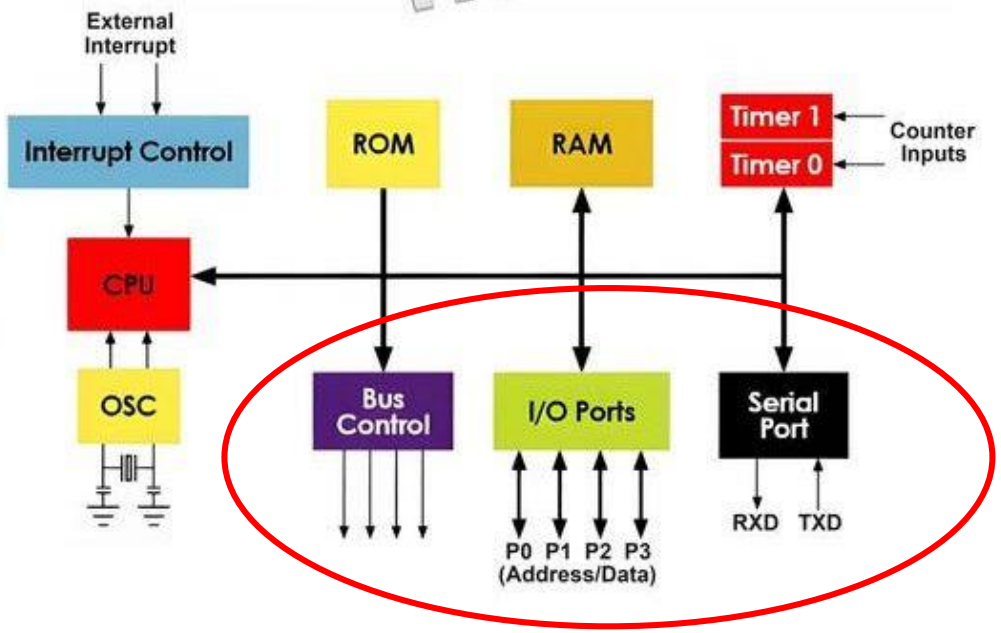
•**Cache memory:** Microprocessors and CPUs use cache memory, a high-speed form of memory located close to the CPU, to store frequently accessed data to accelerate performance.

•**Processor cores:** Individual processing units within microprocessors are known as cores. Modern processors frequently incorporate multiple cores (dual-core, quad-core) allowing for parallel processing by enabling the performance of multiple tasks simultaneously.

•**I/O modules:** A microprocessor's I/O components are critical for managing the flow of data to and from the CPU, including any additional computer peripherals including networking peripherals such as ethernet ports or WiFi units.

Microprocessor vs Microcontroller





Why Study Microprocessors Using a Microcontroller?

1. **Hands-On Learning**: Microcontrollers provide practical experience with real-world hardware, making abstract concepts more tangible.

2. **Integration of Theory and Practice**: Combines theoretical knowledge of microprocessors with applied skills in embedded systems design.

3. **Industry Relevance**: Microcontrollers are widely used in industries such as robotics, IoT, automotive, and consumer electronics.

4. **Comprehensive Understanding**: Covers the full spectrum of microprocessor functionality, including instruction sets, memory management, and peripheral interfacing.

5. **Cost-Effective Learning**: Microcontrollers are affordable and accessible for learning and experimentation.

6. **Preparation for Modern Applications**: Builds a foundation for understanding advanced embedded systems and real-time applications.

7. **Multidisciplinary Skills**: Develops skills in programming (C, assembly), circuit design, and system integration.

Table of Microprocessors and Microcontrollers Examples

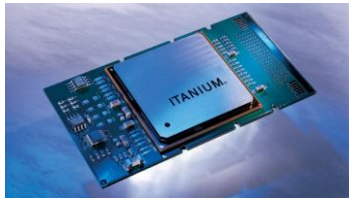
Category	Examples	Description/Applications
Microprocessors	**Intel Core i7**	High-performance computing for PCs and laptops.
	AMD Ryzen 9	Gaming and multitasking applications.
	ARM Cortex-A72	Efficient performance in smartphones and tablets.
	IBM Power9	Enterprise servers and high-performance computing systems.
	Apple M2	System-on-chip with high-performance processing for Apple devices.
Microcontrollers	**Arduino Uno (ATmega328P)**	Educational projects and prototyping.
	ESP32	IoT applications with built-in Wi-Fi and Bluetooth.
	Texas Instruments MSP430	Low-power embedded systems.
	PIC16F877A	Industrial and home automation projects.
	STM32F4	Robotics and advanced embedded systems.

What is Architecture?

Architecture refers to the conceptual design and organization of a processor, defining its instruction set, hardware components, and interaction with software to execute tasks efficiently.

Examples of Processor Architectures

Architecture	Developer	Type	Applications	Examples
x86/x64	Intel & AMD	CISC	PCs, laptops, servers	Intel Core, AMD Ryzen
ARM	ARM Holdings	RISC	Smartphones, IoT, embedded systems	ARM Cortex (A, R, M)
RISC-V	UC Berkeley	RISC	IoT, research, low-cost systems	SiFive processors
MIPS	MIPS Technologies	RISC	IoT, routers, older gaming consoles	Embedded processors, PlayStation 2
PowerPC	IBM, Motorola, Apple	RISC	Consoles (Xbox 360, PS3), servers	IBM Power series
SPARC	Sun Microsystems	RISC	Servers, workstations	Oracle SPARC T5
Itanium (IA-64)	Intel	EPIC	Enterprise servers (now obsolete)	Itanium processors
Z/Architecture	IBM	CISC	Mainframes for large-scale data processing	IBM z15 processors



Why Choose ARM Cortex?

1. **Energy Efficiency**: Industry-leading low power consumption, ideal for mobile and IoT devices.

2. **Versatility**: Wide range of cores (Cortex-A, Cortex-R, Cortex-M) for applications from smartphones to embedded systems.

3. **Scalability**: Supports single-core to multi-core designs for diverse performance needs.

4. **Extensive Ecosystem**: Robust development tools, community support, and compatibility with various platforms.

5. **Cost-Effective**: Affordable licensing and customizable designs for manufacturers.

These advantages make ARM Cortex a top choice for both high-performance and low-power applications.

Software Used in the Course:

• **Cpulator Simulator**: For developing and debugging assembly and C programs for microcontrollers.

•**Wokwi:** An online simulator for microcontrollers, ideal for experimenting with assembly language and C programming, including GPIO, ADC, PWM, and peripheral emulation.
https://wokwi.com/

• **Keil µVision:** A professional-grade IDE for ARM Cortex-M microcontrollers, used for programming, simulation, and debugging in both C and assembly language.

• **STM32CubeIDE (Optional):** A versatile development environment for STM32 microcontrollers, suitable for those interested in real-time applications and advanced debugging.

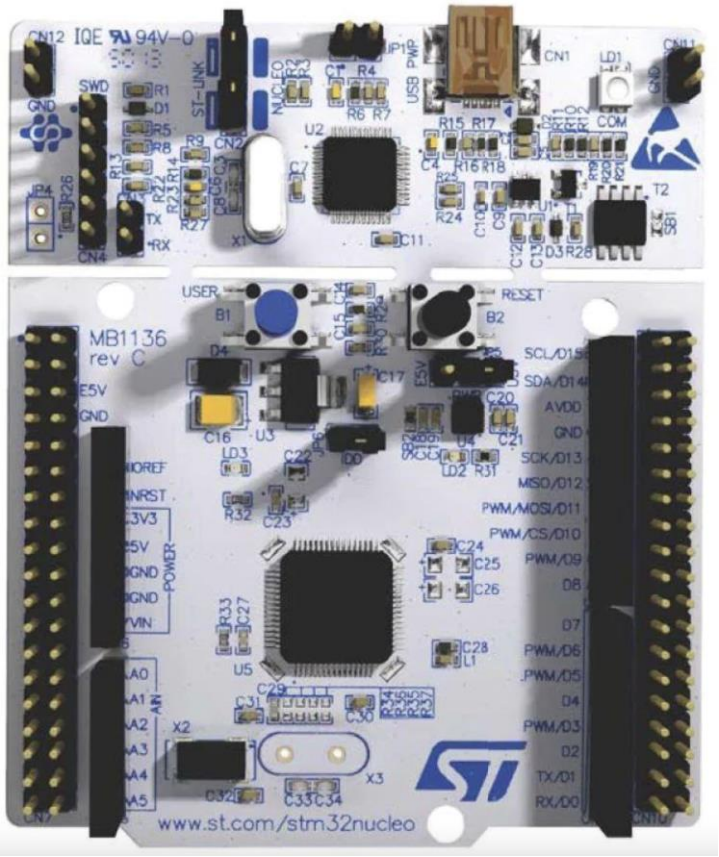
These tools will enable students to simulate, program, and debug microcontroller-based systems efficiently, bridging the gap between theoretical concepts and practical applications.

STM32F446RE Microcontroller

We will use the **STM32F446RE** development board for this course. This microcontroller is part of the ARM Cortex-M4 family, offering advanced capabilities for embedded systems development. It is equipped with:

- **180 MHz CPU frequency**, ensuring high-performance operations.
- **128 KB RAM** and **512 KB Flash memory**, suitable for complex applications.
- Advanced peripherals like **GPIO, ADC, timers, PWM, UART**, and **I2C**, allowing flexible interfacing and control.

The **STM32F446RE** is an affordable and powerful platform for learning microprocessor systems, with a cost of approximately **16-25 USD**. Its versatility makes it ideal for both simulation and real-world embedded applications.



Note: Purchasing this board is optional. Students can complete most of the coursework using simulation tools such as **Wokwi**. For those unable to buy the board, a limited number of **STM32F446RE** boards will be available for loan during the semester. Please plan ahead to ensure access.

Simulation and Practical Use with ST Nucleo Boards

For simulation purposes, we will use the:

- ST Nucleo C031C6 and
- ST Nucleo L031K6 boards.

These boards are fully supported in the Wokwi simulator, allowing students to practice and experiment with embedded systems in a virtual environment.

Additionally, these boards can also be used for real-world practice, enabling hands-on learning and bridging the gap between simulation and hardware implementation.

For real-world practice, students can use these boards or the **STM32F446RE** development board.

Both options provide excellent opportunities for hands-on learning, and programs developed in the simulator can be easily migrated to the STM32F446RE for hardware implementation.

Motivation Example in C

```c
#include "stm32l0xx.h"  // Include the appropriate header file for STM32L031K6

// Function prototypes
void SystemClock_Config(void);
void USART2_Init(void);
void USART2_Write(char *data);

int main(void) {
    // Configure the system clock
    SystemClock_Config();

    // Initialize USART2 for serial communication
    USART2_Init();

    // Send "Hello World" repeatedly
    while (1) {
        USART2_Write("Hello World\r\n");
        for (volatile uint32_t i = 0; i < 500000; i++);  // Delay loop
    }
}

// Function to configure the system clock
void SystemClock_Config(void) {
    // Use the internal 16 MHz oscillator (HSI) as the system clock
    RCC->CR |= RCC_CR_HSION; // Enable HSI
    while (!(RCC->CR & RCC_CR_HSIRDY)); // Wait until HSI is ready

    RCC->CFGR |= RCC_CFGR_SW_HSI; // Set HSI as the system clock source
    while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_HSI); // Wait for HSI to be used as the system clock
}

// Function to initialize USART2
void USART2_Init(void) {
    // Enable GPIOA and USART2 clocks
    RCC->IOPENR |= RCC_IOPENR_GPIOAEN; // Enable GPIOA clock
    RCC->APB1ENR |= RCC_APB1ENR_USART2EN; // Enable USART2 clock

    // Configure PA2 (TX) and PA3 (RX) for alternate function (AF4)
    GPIOA->MODER &= ~((3 << (2 * 2)) | (3 << (3 * 2))); // Clear mode bits
    GPIOA->MODER |= (2 << (2 * 2)) | (2 << (3 * 2)); // Set to alternate function
    GPIOA->AFR[0] |= (4 << (4 * 2)) | (4 << (4 * 3)); // Set AF4 for PA2 and PA3

    // Configure USART2
    USART2->BRR = 16000000 / 9600; // Set baud rate to 9600 (assuming 16 MHz clock)
    USART2->CR1 = USART_CR1_TE | USART_CR1_RE | USART_CR1_UE; // Enable TX, RX, and USART
    while (!(USART2->ISR & USART_ISR_TEACK)); // Wait for TX to be ready
    while (!(USART2->ISR & USART_ISR_REACK)); // Wait for RX to be ready
}

// Function to write a string via USART2
void USART2_Write(char *data) {
    while (*data) {
        while (!(USART2->ISR & USART_ISR_TXE)); // Wait until TX buffer is empty
        USART2->TDR = *data++; // Send character
    }
}
```